

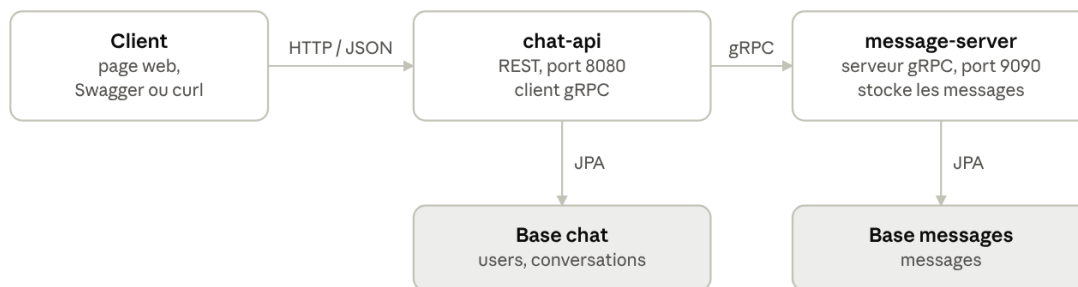
Messaging App – Architecture

Une messagerie en deux microservices Java / Spring Boot : un service REST pour les utilisateurs et les conversations, un serveur gRPC pour les messages, chacun avec sa propre base.

Github : github.com/MegaïSa/OO-project.

Architecture

Deux microservices, chacun avec sa base : REST dehors, gRPC dedans



architecture · 2 services, 2 bases

Le client ne parle qu'à `chat-api` en REST. `chat-api` appelle `message-server` en gRPC, et chaque service n'accède qu'à sa propre base :

Client → Web service REST (client RPC) → serveur RPC, un SGBD par service.

Les modules

Le dépôt est un projet Gradle avec un dossier par module.

Module	Rôle	Communication	Base de données
messaging- <code>proto</code>	Contrat gRPC partagé (<code>message_service.proto</code>), le code Java en est généré	–	–
<code>chat-api</code>	Point d'entrée : utilisateurs, conversations, envoi et lecture des messages	REST vers le client, client gRPC vers <code>message-server</code>	<code>app_user</code> , <code>conversation</code>
<code>message-server</code>	Enregistre et renvoie les messages	Serveur gRPC	<code>messages</code>

Le contrat gRPC définit trois appels : `SendMessage` (envoi), `GetHistory` (historique renvoyé en streaming, message par message) et `CountMessages` (nombre de messages).

Trajet d'un message

Un envoi traverse les deux services en cinq étapes.

1. Le client appelle `POST /conversations/{id}/messages` sur `chat-api`.
2. `chat-api` vérifie dans sa base que la conversation et l'expéditeur existent (sinon 404) et que l'expéditeur en fait partie (sinon 403).
3. `chat-api` appelle `SendMessage` en gRPC sur `message-server`.
4. `message-server` valide le contenu, l'enregistre dans sa base et renvoie le message créé.
5. `chat-api` répond `201 Created` au client, avec le message en JSON.

Si `message-server` est arrêté, les conversations restent consultables (sans le nombre de messages) et l'envoi renvoie une erreur 503 claire au lieu de planter.

Bonnes pratiques

Exigence de l'énoncé	Mise en œuvre
Un dossier par service	<code>messaging-proto/</code> , <code>chat-api/</code> , <code>message-server/</code>
Couches web / services / entités	Packages web (ou gRPC), service, repository, entity dans chaque service
Inversion de contrôle	Composants Spring injectés par constructeur avec <code>@Autowired</code> , stub gRPC injecté par <code>@GrpcClient</code>
Logger au lieu de <code>System.out</code>	SLF4J dans toutes les classes
Exception handler HTTP avec messages	<code>GlobalExceptionHandler</code> : 400, 403, 404, 409, 503 avec un message JSON explicite
Web service stateless	Aucune session ni état en mémoire, tout est en base

Lancer le projet

Prérequis : JDK 21. Les bases H2 sont en mémoire, rien d'autre à installer.

```
./gradlew :message-server:bootRun # terminal 1, gRPC sur le port 9090
./gradlew :chat-api:bootRun        # terminal 2, REST sur le port 8080
```

Ensuite : interface web sur `http://localhost:8080`, documentation Swagger sur

`http://localhost:8080/swagger-ui.html`. Avec Docker, `docker compose up --build` lance les deux services avec une base PostgreSQL chacun.